



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

Departamento de Informática

PROYECTO



Manual Técnico

Autor: Claudio Terrados Sánchez

Curso Académico: 2025/2026

Índice

1. Introducción	3
2. Arquitectura de la aplicación	4
2.1. Frontend	5
2.1.1. Tecnologías usadas	5
2.1.2. Entorno de desarrollo	7
2.2. Backend	8
2.2.1. Tecnologías usadas	8
2.2.2. Entorno de desarrollo	10
3. Documentación técnica	11
3.1. Análisis	11
3.2. Desarrollo	14
3.3. Pruebas realizadas	17
4. Proceso de despliegue	19
5. Propuesta de mejoras	22
6. Bibliografía	24

1. Introducción

AgroControl es un sistema web completo desarrollado para la gestión del control de calidad y la trazabilidad en una fábrica de procesamiento de tomate industrial. La aplicación permite registrar, evaluar y almacenar de forma centralizada los controles de calidad realizados sobre las cargas de tomate que llegan a la fábrica procedentes de distintos proveedores, garantizando que únicamente las materias primas que cumplen los criterios industriales pasen al proceso de transformación.

Durante la campaña del tomate, las fábricas reciben diariamente decenas de camiones cargados de producto. Hasta ahora, el control de calidad se realizaba en muchos casos sobre papel o sobre hojas de cálculo sueltas, lo que dificultaba la consulta posterior de los datos, la trazabilidad por lotes y la elaboración de informes. AgroControl resuelve este problema sustituyendo el proceso manual por un flujo de trabajo digital, multiusuario y con evaluación automática del resultado.

El sistema implementa los siguientes módulos funcionales:

- **Autenticación y control de acceso:** los usuarios acceden mediante email y contraseña. Existen dos roles diferenciados, operario y administrador, con permisos distintos sobre el sistema.
- **Registro de controles de calidad:** el operario puede registrar un nuevo control indicando el camión, el proveedor, la variedad y las mediciones realizadas (kilos totales, kilos sin defectos, kilos verdes, podridos, limitados y nivel de Brix).
- **Evaluación automática de calidad:** al guardar un control, el sistema aplica de forma automática los criterios industriales definidos (defectos totales $\leq 12\%$, podridos $\leq 4\%$, Brix $\geq 4,5$) y determina si el lote queda aprobado o rechazado.
- **Histórico y filtrado:** todos los controles realizados quedan almacenados en la base de datos y se pueden consultar posteriormente aplicando filtros por fecha, proveedor o resultado.
- **Gestión de datos maestros:** las páginas de proveedores, camiones y variedades permiten dar de alta, modificar y desactivar los registros correspondientes.
- **Dashboard de estadísticas:** la pantalla de inicio muestra de forma resumida el número de controles realizados hoy, los camiones procesados y el porcentaje de aprobados.
- **Gestión de usuarios:** solo accesible para administradores, permite crear nuevos operarios, modificar sus datos o desactivar cuentas sin perder la trazabilidad de los controles ya registrados.

La aplicación está diseñada como una arquitectura cliente-servidor en tres capas: una capa de presentación implementada con React, una capa de lógica de negocio y acceso a datos implementada con Node.js y Express, y una capa de persistencia basada en MariaDB. La separación clara entre frontend y backend, así como el uso de una API REST con autenticación basada en JSON Web Tokens, permite que el sistema sea escalable y que pueda ampliarse en el futuro con nuevos clientes (por ejemplo, una aplicación móvil para los operarios de báscula).

2. Arquitectura de la aplicación

AgroControl sigue una arquitectura cliente-servidor de tres capas, donde el frontend (cliente) y el backend (servidor) son aplicaciones independientes que se comunican mediante una API REST sobre HTTP. La base de datos MariaDB constituye la capa de persistencia. Esta separación permite que cada capa evolucione de forma independiente y facilita el mantenimiento y las futuras ampliaciones del sistema.

La estructura de carpetas del proyecto completo es la siguiente:

```
AgroControl/
├── database/                Scripts SQL para MariaDB
│   ├── 01a_schema_sin_trigger.sql
│   ├── 02_seed.sql
│   └── README_BD.md
├── backend/                API REST con Node.js + Express
│   ├── src/
│   │   ├── config/        Configuración de la conexión a la BD
│   │   ├── controllers/   Lógica de negocio de cada recurso
│   │   ├── middlewares/   Middlewares de autenticación JWT
│   │   ├── routes/        Definición de los endpoints
│   │   ├── utils/         Funciones auxiliares (evaluación RF4)
│   │   └── app.js          Punto de entrada del servidor
│   ├── scripts/           Scripts auxiliares
│   ├── .env               Variables de entorno
│   └── package.json
├── frontend/              Interfaz web con React + Vite
│   ├── public/            Archivos estáticos (logo, favicon)
│   ├── src/
│   │   └── api/            Configuración de Axios y servicios
```

Proyecto "Desarrollo de Aplicaciones Web"

Título del Proyecto: AgroControl



			components/	Componentes reutilizables (Layout, Sidebar...)
			context/	Contexto de autenticación
			pages/	Páginas de la aplicación
			styles/	Variables CSS globales
			App.jsx	Definición de rutas
			main.jsx	Punto de entrada
			package.json	
			README.md	Documentación general
			DESIGN_SYSTEM.md	Guía de diseño basada en Figma

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP en formato JSON. El backend expone un conjunto de endpoints RESTful bajo el prefijo /api/, y el frontend los consume a través de la librería Axios. Todas las peticiones protegidas incluyen un token JWT en la cabecera Authorization que el backend valida en cada petición mediante un middleware específico.

2.1. Frontend

El frontend es una aplicación SPA (Single Page Application) desarrollada con React. Toda la interfaz de usuario está construida con componentes reutilizables y emplea React Router para la navegación entre páginas sin recargar el navegador. El diseño visual se ha realizado siguiendo los wireframes de alta fidelidad creados previamente en Figma.

2.1.1. Tecnologías usadas

Las tecnologías y librerías utilizadas en el frontend, con sus versiones exactas, son las siguientes:

Tecnología	Versión	Uso en el proyecto
React	18.2.0	Librería principal para construir la interfaz mediante componentes.
Vite	5.1.0	Bundler y servidor de desarrollo de la SPA.
React Router DOM	6.22.1	Enrutado entre páginas sin recargar el navegador.
Axios	1.6.7	Cliente HTTP para consumir la API REST del backend.
Lucide React	0.344.0	Librería de iconos SVG (sidebar, botones, resultados).
CSS3	Estándar	Estilos personalizados con variables CSS, sin framework.

Justificación de las elecciones realizadas:

- **React frente a Vue o Angular:** se ha optado por React por ser la librería con mayor cuota de mercado en el desarrollo web actual, lo que se traduce en una gran comunidad, abundante documentación y oportunidades laborales. Además, el ciclo formativo ha trabajado con esta tecnología, lo que reduce la curva de aprendizaje.
- **Vite frente a Create React App:** Vite ofrece tiempos de arranque y de recarga en caliente muy superiores gracias al uso de módulos ES nativos del navegador en desarrollo. Create React App está actualmente en mantenimiento mínimo, por lo que Vite es la opción recomendada para nuevos proyectos.

-
- **CSS puro frente a un framework como Tailwind o Bootstrap:** dado que los wireframes en Figma definían un sistema de diseño propio bastante concreto (chips/pills, paleta de colores específica, botones negros), un framework genérico habría supuesto sobrescribir muchos estilos. Con CSS variables y archivos por componente se ha conseguido un control total sobre el aspecto final.
 - **Axios frente a fetch:** Axios permite configurar interceptores de petición y respuesta, lo que se ha usado para añadir automáticamente el token JWT a todas las peticiones y para gestionar de forma centralizada los errores 401 (token expirado).

2.1.2. Entorno de desarrollo

El entorno de desarrollo del frontend se ha configurado sobre los siguientes elementos:

- **Sistema operativo:** Windows 10/11.
- **Editor de código:** Visual Studio Code 1.85+ con las extensiones ES7+ React/Redux/React-Native snippets, Prettier y Auto Rename Tag.
- **Node.js:** v20.x (LTS) como entorno de ejecución de Vite y para el gestor de paquetes npm.
- **Navegador:** Google Chrome con las React Developer Tools instaladas, para inspeccionar componentes, props y estado.
- **Servidor de desarrollo:** Vite arrancado mediante `npm run dev` sirve la aplicación en `http://localhost:5173` con recarga en caliente.
- **Figma:** se ha empleado para diseñar los wireframes de alta fidelidad de todas las pantallas antes de empezar a codificar (login, dashboard, nuevo control, historial).
- **Control de versiones:** Git, con GitHub Desktop como cliente gráfico y GitHub como repositorio remoto.

Estructura interna del frontend:

```
frontend/src/
├── api/
│   ├── axios.js          Configuración global de Axios e interceptores
│   └── services.js        Servicios por recurso (auth, controles, etc.)
├── components/
│   ├── layout/            Layout, Sidebar y Header con sus CSS
│   └── ProtectedRoute.jsx Componente para rutas privadas
├── context/
│   └── AuthContext.jsx     Contexto global de autenticación
├── pages/                 Una página por funcionalidad
│   ├── Login.jsx + Login.css
│   ├── Dashboard.jsx + Dashboard.css
│   ├── NuevoControl.jsx + NuevoControl.css
│   ├── Historial.jsx + Historial.css
│   ├── Proveedores.jsx / Camiones.jsx / Variedades.jsx / Usuarios.jsx
│   └── Gestion.css         Estilos compartidos de páginas CRUD
```


	└─ Perfil.jsx + Perfil.css	
	└─ styles/	
	└─ variables.css	Variables CSS globales (colores, tipografía)
	└─ globals.css	Reset y estilos base
	└─ App.jsx	Definición de rutas con React Router
	└─ main.jsx	Punto de entrada y montaje en el DOM

2.2. Backend

El backend es una API REST construida sobre Node.js y Express. Es totalmente sin estado (stateless): cada petición HTTP debe incluir el token JWT del usuario que la realiza para que el servidor pueda autorizarla. La base de datos MariaDB se accede mediante el driver mysql2 con un pool de conexiones, lo que permite gestionar peticiones concurrentes de forma eficiente.

2.2.1. Tecnologías usadas

La pila tecnológica del backend, con las versiones exactas instaladas, es la siguiente:

Tecnología	Versión	Uso en el proyecto
Node.js	20.x LTS	Entorno de ejecución de JavaScript del lado del servidor.
Express	4.18.2	Framework web para definir las rutas y middlewares.
MariaDB	10.6+	Sistema gestor de base de datos relacional.
mysql2	3.9.1	Driver de Node.js para conectar con MariaDB/MySQL.
bcrypt	5.1.1	Cifrado de contraseñas con 10 rondas de salt.
jsonwebtoken	9.0.2	Generación y verificación de tokens JWT.
cors	2.8.5	Middleware para permitir peticiones cross-origin desde el frontend.
dotenv	16.4.5	Carga las variables de entorno desde un archivo .env.
nodemon	3.0.3 (dev)	Reinicio automático del servidor en desarrollo.

Justificación de las elecciones realizadas:

- **Node.js + Express frente a PHP o Java:** el uso de JavaScript tanto en el frontend como en el backend reduce la fricción cognitiva entre ambas capas, permite reutilizar conocimiento sobre el lenguaje y simplifica el intercambio de datos (todo es JSON). Express es el framework web más utilizado del ecosistema Node y se enseña en el ciclo.
- **MariaDB frente a MySQL o PostgreSQL:** MariaDB es un fork libre y totalmente compatible con MySQL, mantenido por los creadores originales del proyecto. Se ha trabajado con él durante el ciclo y ofrece todas las funcionalidades necesarias (claves foráneas, CHECK constraints, columnas virtuales calculadas).
- **mysql2 frente a Sequelize u otros ORMs:** para el alcance del proyecto se ha preferido escribir las consultas SQL directamente, aprovechando las consultas preparadas que ofrece mysql2 como protección frente a SQL injection. Esto evita una capa adicional de abstracción y permite mostrar de forma explícita las uniones (JOIN) entre tablas.
- **JWT frente a sesiones con cookies:** los JWT permiten que la API sea totalmente stateless: el servidor no necesita guardar información de sesión, lo que simplifica el despliegue y el escalado horizontal. El token incluye el id del usuario, su rol y su fecha de expiración (8 horas, equivalente a una jornada laboral).
- **bcrypt frente a SHA o MD5:** bcrypt es el algoritmo recomendado para almacenar contraseñas, ya que incluye salt automático y permite ajustar el coste computacional para resistir ataques de fuerza bruta. SHA y MD5 son funciones hash rápidas, lo que las hace inadecuadas para este uso.

2.2.2. Entorno de desarrollo

El entorno de desarrollo del backend comparte la misma máquina y editor que el del frontend, pero añade herramientas específicas para la base de datos y para probar la API:

- **Sistema operativo:** Windows 10/11.
- **Editor de código:** Visual Studio Code 1.85+.
- **Node.js:** v20.x con npm como gestor de paquetes.
- **Servidor MariaDB:** 10.6+ instalado de forma local, escuchando en el puerto por defecto 3306.
- **HeidiSQL:** cliente gráfico para gestionar la base de datos. Se ha utilizado para ejecutar los scripts iniciales (creación del esquema y carga de datos de prueba) y para verificar el estado de las tablas durante el desarrollo.
- **Thunder Client:** extensión de VS Code utilizada como cliente HTTP para probar manualmente todos los endpoints de la API. Permite enviar peticiones con distintos métodos, configurar cabeceras (incluyendo Authorization Bearer) y guardar colecciones de peticiones.
- **Variables de entorno:** archivo .env con las credenciales de la base de datos, la clave secreta de JWT y el puerto del servidor. Este archivo está incluido en el .gitignore y nunca se sube al repositorio.

Estructura interna del backend:

```
backend/  
├─ src/  
│   ├─ app.js                Punto de entrada Express  
│   ├─ config/  
│   │   └─ database.js       Pool de conexiones a MariaDB  
│   ├─ utils/  
│   │   └─ evaluarCalidad.js Algoritmo RF4  
│   ├─ middlewares/  
│   │   └─ auth.js           Validación de JWT y roles  
│   ├─ controllers/          Lógica de negocio por recurso  
│   │   ├─ auth.controller.js  
│   │   ├─ controles.controller.js  
│   │   ├─ proveedores.controller.js  
│   │   └─ camiones.controller.js
```

			variedades.controller.js	
			usuarios.controller.js	
			routes/	Definición de endpoints REST
			(mismos nombres .routes.js)	
			scripts/	
			generar-hashes.js	Inicializa contraseñas de los usuarios seed
			.env	Variables de entorno (no se sube a Git)
			.env.example	Plantilla del .env
			package.json	

3. Documentación técnica

En este apartado se recogen los diagramas y modelos elaborados durante las fases de análisis y diseño del proyecto, así como las pruebas realizadas sobre el sistema.

3.1. Análisis

Durante la fase de análisis se identificaron los actores del sistema, los casos de uso principales y se modelaron las entidades de datos. A continuación, se describen estos elementos.

Actores del sistema

El sistema cuenta con dos actores principales, ambos usuarios autenticados:

- **Operario:** es el usuario que se encuentra a pie de báscula o en el laboratorio. Su tarea principal es registrar los controles de calidad de los camiones que llegan a la fábrica y consultar el histórico. No puede gestionar datos maestros ni dar de alta usuarios.
- **Administrador:** es el responsable del sistema. Además de todo lo que puede hacer un operario, gestiona los datos maestros (proveedores, camiones, variedades) y crea/desactiva cuentas de usuario.

Casos de uso principales

Los casos de uso más relevantes identificados son los siguientes:

- **CU-01 Iniciar sesión:** el usuario introduce su email y contraseña; si las credenciales son válidas y la cuenta está activa, el sistema genera un JWT y le permite acceder al resto de funcionalidades.

- **CU-02 Registrar nuevo control:** el operario selecciona proveedor, camión, variedad e introduce las mediciones. Al guardar, el sistema evalúa el resultado automáticamente y almacena el registro.
- **CU-03 Consultar historial:** cualquier usuario autenticado puede ver todos los controles registrados y filtrar por fecha, proveedor o resultado (aprobado, rechazado o pendiente).
- **CU-04 Gestionar proveedores / camiones / variedades:** el administrador puede crear, editar y desactivar registros maestros.
- **CU-05 Gestionar usuarios:** el administrador puede crear nuevos operarios o administradores, modificar sus datos o desactivar cuentas. Las cuentas nunca se eliminan físicamente, solo se desactivan (soft delete).
- **CU-06 Consultar dashboard:** al iniciar sesión, todos los usuarios ven un resumen con los controles del día, los camiones procesados y el porcentaje de aprobados.

Diagrama entidad-relación

El modelo de datos consta de seis entidades principales. La entidad central del sistema es control_calidad, que registra cada medición realizada sobre una carga de tomate concreta. Está relacionada con el usuario que lo registra, con el camión que transporta la carga y con la variedad evaluada.

Entidades del modelo:

Entidad	Descripción y atributos principales
USUARIO	Operarios y administradores. id_usuario (PK), nombre, email (único), password_hash, rol (operario/administrador), activo, fecha_creacion.
PROVEEDOR	Cooperativas y empresas suministradoras de tomate. id_proveedor (PK), nombre (único), localidad, activo.
CAMION	Camiones que transportan la materia prima. id_camion (PK), matricula (única), id_proveedor (FK), activo.
VARIEDAD	Variedades de tomate industrial (Heinz 1015, Rio Grande, H-9661, CXD-179, Kalvert...). id_variedad (PK), nombre_variedad (único), activo, fecha_creacion.

Entidad	Descripción y atributos principales
PROVEEDOR_VARIEDAD	Tabla intermedia N:M para indicar qué variedades cultiva cada proveedor. (id_proveedor, id_variedad) como PK compuesta.
CONTROL_CALIDAD	Cada control realizado. id_control (PK), id_usuario (FK), id_camion (FK), id_variedad (FK), fecha_control, kilos_total, kilos_sin_defectos, kilos_verde, kilos_podridos, kilos_limitado, porcentaje_defectos (columna virtual), brix, resultado (aprobado/rechazado/pendiente), observaciones.

Las relaciones principales son:

- Un USUARIO registra muchos CONTROL_CALIDAD (1:N).
- Un CAMION es controlado en muchos CONTROL_CALIDAD (1:N).
- Una VARIEDAD aparece en muchos CONTROL_CALIDAD (1:N).
- Un PROVEEDOR posee muchos CAMION (1:N).
- Un PROVEEDOR cultiva muchas VARIEDAD y una VARIEDAD puede ser cultivada por varios PROVEEDOR (N:M, resuelta con la tabla PROVEEDOR_VARIEDAD).

Reglas de negocio y restricciones de integridad

Se han definido las siguientes reglas y restricciones, implementadas tanto en la base de datos como en el backend:

- **Las contraseñas nunca se guardan en claro:** se almacena únicamente el hash bcrypt en la columna password_hash de la tabla usuario.
- **Los emails son únicos en toda la aplicación:** no pueden existir dos usuarios con el mismo email.
- **Las matrículas de camión son únicas:** se evita que un mismo camión esté duplicado en el sistema.
- **Las claves foráneas usan ON DELETE RESTRICT:** no se permite eliminar físicamente un usuario, un camión o una variedad mientras tenga controles asociados, para preservar la trazabilidad.
- **Restricciones CHECK sobre control_calidad:** los kilos deben ser positivos, la suma de las partes (sin defectos, verdes, podridos, limitados) no puede superar los kilos totales, y el Brix debe estar en el rango 0-15.
- **Soft delete en lugar de borrado físico:** la columna activo permite desactivar registros sin perderlos. Las consultas por defecto filtran por activo = TRUE.
- **Evaluación automática del resultado:** el campo resultado se calcula por el backend al insertar el control, aplicando los criterios industriales descritos en el siguiente apartado.

3.2. Desarrollo

Modelo lógico de la base de datos

A partir del diagrama E/R se ha derivado el modelo lógico, donde las relaciones N:M se convierten en una tabla intermedia y los atributos calculados se implementan mediante columnas virtuales. Las claves primarias se han definido como enteros autoincrementales (AUTO_INCREMENT).

Tablas y claves principales (notación: PK = clave primaria, FK = clave foránea, U = único):

```
usuario(id_usuario PK, nombre, email U, password_hash, rol,  
        activo, fecha_creacion)  
proveedor(id_proveedor PK, nombre U, localidad, activo)  
camion(id_camion PK, matricula U,  
        id_proveedor FK -> proveedor, activo)
```

```
variedad(id_variedad PK, nombre_variedad U, activo, fecha_creacion)
proveedor_variedad(id_proveedor FK PK, id_variedad FK PK)
control_calidad(id_control PK,
                id_usuario FK -> usuario,
                id_camion FK -> camion,
                id_variedad FK -> variedad,
                fecha_control, kilos_total,
                kilos_sin_defectos, kilos_verde,
                kilos_podridos, kilos_limitado,
                porcentaje_defectos VIRTUAL,
                brix, resultado ENUM, observaciones)
```

Criterios de evaluación automática (RF4)

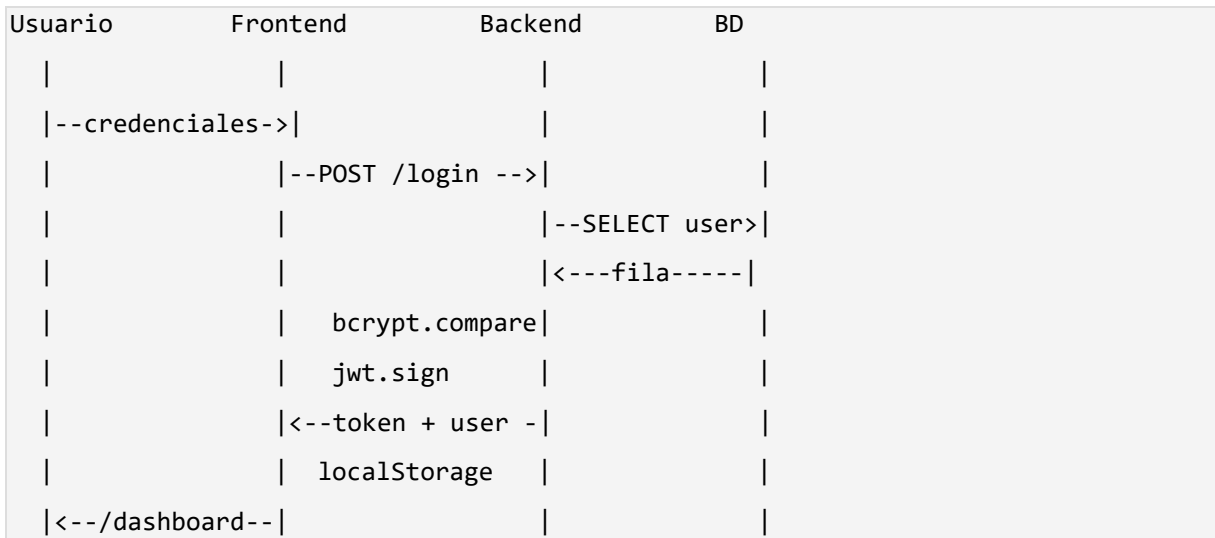
La función evaluarCalidad implementada en backend/src/utills/evaluarCalidad.js aplica los siguientes criterios sobre cada control para determinar su resultado:

- **Defectos totales ≤ 12 %:** la suma de kilos verdes, podridos y limitados, dividida entre los kilos totales, no debe superar el 12 %.
- **Podridos ≤ 4 %:** como umbral más estricto por el riesgo microbiológico, los kilos podridos no pueden representar más del 4 % de la carga.
- **Brix $\geq 4,5$:** el grado Brix mínimo exigido para que el tomate sea apto para la elaboración de concentrado.

Si los tres criterios se cumplen, el resultado del control es aprobado; si falla alguno, el resultado es rechazado. El operario puede añadir observaciones en texto libre.

Diagrama de secuencia: inicio de sesión

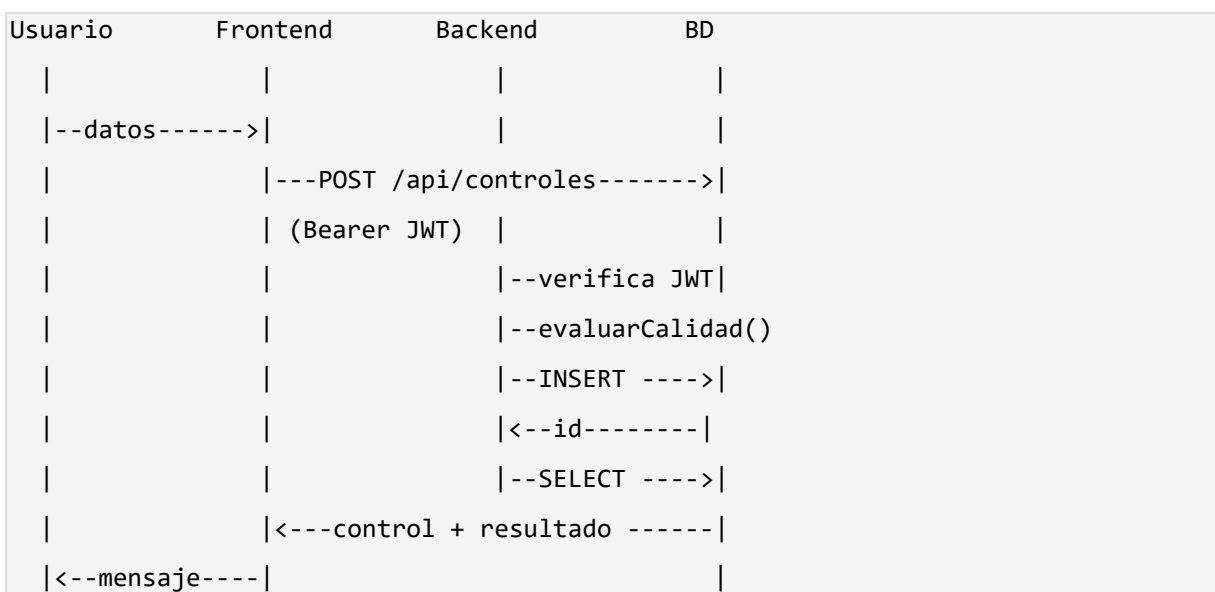
El siguiente diagrama de secuencia describe el flujo del caso de uso CU-01 (iniciar sesión):



Cuando el usuario envía sus credenciales, el frontend hace una petición POST a /api/auth/login con email y password en el body. El backend busca el usuario en la base de datos, compara la contraseña con bcrypt.compare y, si es correcta, genera un JWT firmado con la clave secreta. El token se devuelve al frontend, que lo guarda en localStorage y redirige al usuario al dashboard.

Diagrama de secuencia: registrar un nuevo control

Para el caso de uso CU-02 (registrar nuevo control) el flujo es el siguiente:



El frontend envía todos los datos del control (camión, variedad, kilos, brix...) junto con el token JWT en la cabecera Authorization. El middleware de autenticación valida el token, después el controlador llama a evaluarCalidad para calcular si el lote queda aprobado o rechazado, e inserta el registro en la tabla control_calidad. Por último, se devuelve al frontend el control completo con su resultado, y la interfaz muestra un mensaje y redirige al historial.

Endpoints REST implementados

La API expone los siguientes endpoints principales bajo el prefijo /api/:

Método	Ruta	Auth	Rol
POST	/auth/login	No	—
POST	/auth/register	Sí	Administrador
GET	/auth/perfil	Sí	Cualquiera
GET	/controles?fecha=&proveedor=&resultado=	Sí	Cualquiera
POST	/controles	Sí	Cualquiera
PUT	/controles/:id	Sí	Cualquiera
DELETE	/controles/:id	Sí	Administrador
GET	/controles/estadisticas/hoy	Sí	Cualquiera
CRUD	/proveedores, /camiones, /variedades, /usuarios	Sí	Lectura: cualquiera; Escritura: admin

3.3. Pruebas realizadas

A lo largo del desarrollo se han ido realizando pruebas tanto sobre el backend como sobre el frontend. Dado el alcance del proyecto, las pruebas son principalmente manuales y se han documentado conforme se ejecutaban. Como herramienta principal se ha utilizado Thunder Client (extensión de VS Code) para las pruebas de la API REST.

Prueba 1: Inicio de sesión con credenciales correctas

- **Componente:** POST /api/auth/login (controllers/auth.controller.js).
- **Herramienta:** Thunder Client.
- **Datos de entrada:** {"email": "admin@agrocontrol.local", "password": "Admin1234!"}.
- **Resultado esperado:** Status 200 OK. Respuesta JSON con mensaje, token (cadena JWT) y objeto usuario con id, nombre, email y rol.
- **Resultado obtenido:** Status 200 OK, tamaño de respuesta 3,14 KB, tiempo 11 ms. Token recibido correctamente y guardado en localStorage por el frontend.  Correcto.

Prueba 2: Acceso a recurso protegido con JWT

- **Componente:** GET /api/controles + middleware verificarToken.
- **Herramienta:** Thunder Client.
- **Datos de entrada:** Cabecera Authorization: Bearer <token JWT obtenido en la prueba 1>.
- **Resultado esperado:** Status 200 OK. Devolver la lista de controles de calidad existentes con todos sus campos y los nombres del usuario, camión, proveedor y variedad relacionados (vía JOIN).
- **Resultado obtenido:** Status 200 OK. JSON con campo total y array controles. Se observa correctamente la información unida de las tablas relacionadas.  Correcto.

Prueba 3: Acceso sin token

- **Componente:** Middleware verificarToken.
- **Herramienta:** Thunder Client.
- **Datos de entrada:** Petición GET /api/controles sin cabecera Authorization.

- **Resultado esperado:** Status 401 Unauthorized con mensaje "Token no proporcionado".
- **Resultado obtenido:** Status 401, mensaje correcto. ☒ Correcto.

Prueba 4: Evaluación automática de calidad (aprobado)

- **Componente:** evaluarCalidad() en utils/evaluarCalidad.js.
- **Datos de entrada:** { kilos_total: 25000, kilos_verde: 800, kilos_podridos: 200, kilos_limitado: 500, brix: 5.2 }.
- **Cálculos:** % defectos = $(800+200+500)/25000 = 6,0\% \leq 12\%$. % podridos = $200/25000 = 0,8\% \leq 4\%$. Brix $5,2 \geq 4,5$.
- **Resultado esperado:** "aprobado".
- **Resultado obtenido:** "aprobado". ☒ Correcto.

Prueba 5: Evaluación automática de calidad (rechazado por podridos)

- **Componente:** evaluarCalidad().
- **Datos de entrada:** { kilos_total: 20000, kilos_verde: 500, kilos_podridos: 1200, kilos_limitado: 300, brix: 5.0 }.
- **Cálculos:** % podridos = $1200/20000 = 6,0\% > 4\%$ (incumple).
- **Resultado esperado:** "rechazado".
- **Resultado obtenido:** "rechazado". ☒ Correcto.

Prueba 6: Validación de campos negativos en /api/controles

- **Componente:** controllers/controles.controller.js (crearControl).
- **Datos de entrada:** POST /api/controles con kilos_total = -100 (valor negativo).
- **Resultado esperado:** Status 400 Bad Request con mensaje "Los kilos no pueden ser negativos y kilos_total debe ser mayor que 0".
- **Resultado obtenido:** Status 400, mensaje correcto. ☒ Correcto.

Prueba 7: Restricción de roles

- **Componente:** Middleware verificarAdmin sobre /api/usuarios.
- **Datos de entrada:** GET /api/usuarios con un token JWT de un usuario con rol operario.

-
- **Resultado esperado:** Status 403 Forbidden con mensaje "Acceso denegado. Se requiere rol de administrador".
 - **Resultado obtenido:** Status 403, mensaje correcto.  Correcto.

Prueba 8: Login del frontend

- **Componente:** Página Login.jsx + contexto AuthContext.
- **Herramienta:** Navegador Google Chrome + React DevTools.
- **Datos de entrada:** Email y contraseña válidos introducidos en el formulario.
- **Resultado esperado:** Redirección a /dashboard, token guardado en localStorage, sidebar visible.
- **Resultado obtenido:** Redirección correcta, token presente en localStorage con la clave agrocontrol_token, dashboard visible con estadísticas.  Correcto.

Prueba 9: CRUD completo de proveedores desde el frontend

- **Componente:** Página Proveedores.jsx.
- **Datos de entrada:** Crear "Proveedor Test" en Badajoz, editar a Cáceres, desactivar.
- **Resultado esperado:** Las tres operaciones se reflejan en la tabla y en la base de datos (verificado con HeidiSQL).
- **Resultado obtenido:** Las operaciones se ejecutan correctamente, los datos persisten al recargar y la fila desactivada muestra el icono  Inactivo.  Correcto.

4. Proceso de despliegue

Este apartado describe paso a paso cómo desplegar la aplicación AgroControl en un equipo local. El proyecto está alojado en GitHub y puede clonarse libremente. La aplicación se ejecuta en modo desarrollo durante el periodo evaluativo del proyecto; en la sección 5 se proponen mejoras para un despliegue en producción mediante hosting en la nube.

Repositorio del proyecto

El código fuente completo está disponible en el siguiente repositorio público de GitHub:

<https://github.com/Claud7io/AgroControl>

4.1. Software necesario

Para desplegar la aplicación en una máquina nueva es necesario instalar previamente el siguiente software, en las versiones indicadas:

Software	Versión	Descarga
Node.js	20.x LTS	https://nodejs.org/
MariaDB Server	10.6+	https://mariadb.org/download/
HeidiSQL	12.x	https://www.heidisql.com/download.php
Git	2.40+	https://git-scm.com/downloads
Visual Studio Code	1.85+	https://code.visualstudio.com/
Thunder Client (ext.)	Última	Marketplace de VS Code (opcional, para probar la API)

4.2. Pasos para el despliegue

Una vez instalado el software anterior, los pasos para poner en marcha la aplicación son los siguientes.

Paso 1: Clonar el repositorio

```
git clone https://github.com/Claud7io/AgroControl.git  
cd AgroControl
```

Paso 2: Crear la base de datos

Abrir HeidiSQL y conectar al servidor MariaDB local (host: localhost, usuario: root, puerto: 3306). Crear una base de datos vacía llamada agrocontrol:

```
CREATE DATABASE agrocontrol  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```

A continuación, abrir y ejecutar los scripts en el siguiente orden, situados en la carpeta database/:

- 01a_schema_sin_trigger.sql — crea las tablas con sus claves primarias, foráneas, restricciones CHECK e índices.
- 02_seed.sql — inserta los datos de prueba: tres usuarios (admin, operario y maría.gomez), cuatro proveedores de la zona de Badajoz, cinco variedades industriales, seis camiones y seis controles variados.

Paso 3: Configurar y arrancar el backend

Desde la carpeta backend/ se instalan las dependencias, se crea el archivo .env con las credenciales reales y se generan los hashes de las contraseñas de prueba:

```
cd backend  
npm install  
copy .env.example .env
```

Editar el archivo .env recién creado e indicar la contraseña de MariaDB y una clave secreta larga para los JWT:

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=tu_password_de_mariadb  
DB_NAME=agrocontrol  
DB_PORT=3306  
JWT_SECRET=cadena_larga_y_aleatoria_para_firmar_tokens  
PORT=3000  
NODE_ENV=development
```

A continuación se ejecuta el script que actualiza las contraseñas de los usuarios seed con hashes bcrypt reales:

Proyecto "Desarrollo de Aplicaciones Web"

Título del Proyecto: AgroControl



JUNTA DE EXTREMADURA

Consejería de Educación y Empleo

```
npm run generar-hashes
```

Por último, se arranca el servidor:

```
npm start
```

Si todo es correcto, aparecerá en consola el mensaje "Conexión exitosa a MariaDB" y "Servidor corriendo en http://localhost:3000". Se puede comprobar abriendo el navegador en esa URL: debe devolver un JSON con la información de la API.

Paso 4: Configurar y arrancar el frontend

En una nueva ventana de terminal, desde la carpeta frontend/, se instalan las dependencias y se arranca el servidor de desarrollo:

```
cd frontend  
npm install  
npm run dev
```

Vite abrirá automáticamente el navegador en <http://localhost:5173>. Aparecerá la pantalla de login.

Paso 5: Acceder a la aplicación

Para entrar en el sistema se pueden utilizar las siguientes credenciales de prueba creadas por el script de seed:

Email	Contraseña	Rol
admin@agrocontrol.local	Admin1234!	Administrador
operario@agrocontrol.local	Operario1234!	Operario
maria.gomez@agrocontrol.local	Operario1234!	Operario

Tras iniciar sesión, el sistema redirige al dashboard, desde donde se accede al resto de funcionalidades a través del menú lateral.

4.3. Resumen de URLs locales

Componente	URL local
Frontend (React + Vite)	http://localhost:5173
Backend (API REST)	http://localhost:3000
Base de datos MariaDB	localhost:3306 (esquema agrocontrol)

4.4. Hosting

En la versión presentada como Trabajo de Fin de Ciclo, la aplicación se ejecuta en local: la base de datos en el servidor MariaDB del propio equipo, el backend con node, y el frontend con Vite. No se ha contratado un hosting durante la entrega, ya que el alcance del proyecto está centrado en la funcionalidad. En la sección 5 (Propuesta de mejoras) se describe cómo se podría desplegar la aplicación en la nube.

5. Propuesta de mejoras

Aunque AgroControl cumple con todos los requisitos funcionales planteados, se han identificado varias líneas de mejora que podrían abordarse en futuras evoluciones del sistema. Se agrupan en mejoras funcionales, técnicas y de despliegue.

5.1. Mejoras funcionales

- **Exportación de informes a PDF y Excel:** permitir al administrador descargar el histórico filtrado en formato PDF (para imprimir) o Excel (para procesar en hojas de cálculo). Se podría implementar con librerías como pdfmake o exceljs en el backend.
- **Gráficos avanzados en el dashboard:** incluir gráficos de evolución del % de aprobados por semana, comparativas entre proveedores y por variedad. Recharts o Chart.js encajarían bien con la base ya construida en React.
- **Notificaciones por email:** enviar un aviso automático al responsable cuando un proveedor acumule un determinado número de cargas rechazadas en un periodo.
- **Sistema de auditoría:** guardar en una tabla aparte cada modificación de un control (quién, cuándo, qué cambió) para reforzar la trazabilidad.
- **Modo offline para báscula:** guardar los controles en local cuando no haya red y sincronizarlos automáticamente al recuperar la conexión, usando una PWA con Service Workers.
- **Multi-idioma:** internacionalizar la interfaz para campañas en las que se trabaje con personal no hispanohablante (ej. inglés, portugués), usando react-i18next.

5.2. Mejoras técnicas

- **Pruebas automatizadas:** introducir Jest o Vitest para escribir tests unitarios sobre la función evaluarCalidad y los controladores del backend, y tests de integración con Supertest. En el frontend, React Testing Library para los componentes principales.
- **CI/CD con GitHub Actions:** automatizar la ejecución de los tests en cada push y publicar automáticamente cada release etiquetada.
- **Documentación interactiva de la API:** añadir Swagger / OpenAPI para que los endpoints estén autodocumentados y puedan probarse desde el navegador.

- **Refresh tokens:** el JWT actual dura 8 horas. Implementar un sistema de refresh token permitiría sesiones más seguras y experiencias más fluidas.
- **Mejora del manejo de errores en el frontend:** centralizar todos los errores en un sistema de notificaciones tipo toast (react-hot-toast) en lugar de usar alert.
- **Modo oscuro:** aprovechando que ya se trabaja con variables CSS, sería sencillo añadir un tema oscuro alternativo.

5.3. Mejoras de despliegue y producción

- **Hosting del frontend en Vercel o Netlify:** ambos servicios ofrecen plan gratuito que conecta directamente con el repositorio de GitHub y despliega automáticamente cada push a la rama main. Son ideales para SPAs como esta.
- **Hosting del backend en Render o Railway:** permiten desplegar aplicaciones Node.js gratuitas con dominio público y HTTPS automático. La base de datos puede ir en el mismo proveedor o en PlanetScale (MySQL serverless).
- **Variables de entorno en producción:** usar el sistema de secrets del proveedor en lugar del archivo .env, evitando filtraciones accidentales.
- **HTTPS y CORS restrictivo:** en producción, el CORS debe limitarse al dominio real del frontend, no permitir cualquier origen.
- **Aplicación móvil con React Native:** los operarios de báscula trabajan a menudo con tablets o móviles. Una app nativa permitiría una mejor experiencia que la versión web, incluyendo lectura de matrículas mediante OCR.

Estas mejoras se han priorizado en función de su valor para el usuario final y de su complejidad técnica. En una segunda iteración del proyecto, las primeras candidatas serían la exportación a PDF/Excel y los gráficos avanzados del dashboard.

6. Bibliografía

Durante el desarrollo de AgroControl se han consultado los siguientes recursos. Las URLs corresponden a la documentación oficial de cada tecnología, lo que garantiza información fiable y actualizada.

Documentación oficial de las tecnologías utilizadas

- **Node.js** — <https://nodejs.org/docs/latest-v20.x/api/>
- **Express** — <https://expressjs.com/es/4x/api.html>
- **React (documentación oficial nueva)** — <https://react.dev/>
- **Vite** — <https://vitejs.dev/guide/>
- **React Router DOM** — <https://reactrouter.com/en/main>
- **Axios** — <https://axios-http.com/docs/intro>
- **MariaDB Knowledge Base** — <https://mariadb.com/kb/en/>
- **mysql2 (Node)** — <https://github.com/sidorares/node-mysql2>
- **bcrypt para Node** — <https://github.com/kelektiv/node.bcrypt.js>
- **jsonwebtoken** — <https://github.com/auth0/node-jwebtoken>
- **Lucide Icons** — <https://lucide.dev/>
- **MDN Web Docs (JS, HTML, CSS)** — <https://developer.mozilla.org/es/>

Artículos y recursos consultados

- **JSON Web Tokens — Introduction** — <https://jwt.io/introduction>
- **Auth0: "What is JWT?"** — <https://auth0.com/learn/json-web-tokens>
- **OWASP: "Password Storage Cheat Sheet"** — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- **Smashing Magazine: "A Complete Guide to React Router"** — <https://www.smashingmagazine.com/>
- **REST API Tutorial** — <https://restfulapi.net/>

Herramientas de desarrollo

- **Visual Studio Code** — <https://code.visualstudio.com/docs>
- **HeidiSQL** — <https://www.heidisql.com/help.php>
- **Thunder Client** — <https://www.thunderclient.com/>

Proyecto "Desarrollo de Aplicaciones Web"

Título del Proyecto: AgroControl



-
- **Figma (diseño de wireframes)** — <https://www.figma.com/>
 - **GitHub Docs** — <https://docs.github.com/es>

Material del ciclo formativo

- **Apuntes y materiales del módulo de Desarrollo Web en Entorno Cliente**, IES Albarregas, curso 2025/2026.
- **Apuntes y materiales del módulo de Desarrollo Web en Entorno Servidor**, IES Albarregas, curso 2025/2026.
- **Apuntes y materiales del módulo de Despliegue de Aplicaciones Web**, IES Albarregas, curso 2025/2026.